

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2
По дисциплине «Обработка изображений
в интеллектуальных системах»
Тема «Конструирование моделей на базе
предобученных нейронных сетей»
Вариант 5

Выполнил
Студент 4 курса
Группы ИИ-25
Дулько Денис Андреевич

Проверил
Андренко К. В.

Цель работы

Научиться адаптировать предобученную нейронную сеть к новой выборке и сравнивать её с собственной архитектурой.

Задание по варианту 5

Адаптировать предобученную DenseNet121 для размеченной части STL-10, использовать SGD, построить график ошибки и оценить точность. Сравнить с CNN из ЛР 1 и опубликованными SOTA-результатами. Реализовать предсказание обеих сетей на одном произвольном изображении.

Данные и подготовка

STL-10 содержит 10 классов цветных изображений 96×96 : самолёт, птица, автомобиль, кошка, олень, собака, лошадь, обезьяна, корабль и грузовик. Используется `torchvision.datasets.STL10`. Из 5000 размеченных обучающих изображений выделены 4000 для обучения и 1000 для валидации: по 400 и 100 изображений каждого класса. Seed равен 5; разделение в обеих лабораторных одинаковое. Неразмеченные данные в обучение не включены.

Полная тестовая выборка из 8000 изображений используется только для итоговой оценки. Лучшие веса выбираются по минимальной ошибке на валидации. Графики показывают ошибку обучения и валидации, поэтому тест не влияет на выбор эпохи.

Архитектура и обучение

DenseNet121 загружается с весами `DenseNet121_Weights.DEFAULT`, обученными на ImageNet. Базовые признаки заморожены; классификатор заменён слоем `Linear 1024 → 10`. Обучаются только 10 250 параметров этого слоя. Модуль `features` работает в режиме `eval`, чтобы не менять статистику `BatchNorm`.

Изображения масштабируются до 256 и обрезаются до 224×224 . На обучении обрезка случайная и применяется горизонтальное отражение, на валидации и тесте обрезка центральная. Нормализация ImageNet: `mean=(0.485, 0.456, 0.406)`, `std=(0.229, 0.224, 0.225)`.

SGD: `lr=0.01`, `momentum=0.9`, `weight_decay=0.0001`; `CosineAnnealingLR` уменьшает шаг обучения. Функция ошибки `CrossEntropyLoss`, `batch_size=64`, число эпох 8. Вычисления выполнены на NVIDIA GeForce RTX 3070, PyTorch 2.8.0+cu128.

Результаты обучения

На полной тестовой выборке точность равна **96.40%** средняя ошибка CrossEntropyLoss **0.1098**. Выбрана эпоха 7 по ошибке валидации. Число параметров модели 6 964 106; обучаемых 10 250. Время обучения и итоговой оценки 1.7 мин.

Эпоха	Train loss	Val loss	Train acc %	Val acc %
1	0.4301	0.1039	87.58	97.40
2	0.1145	0.0953	96.55	97.20
3	0.0923	0.0891	97.15	97.50
4	0.0819	0.0872	97.60	97.60
5	0.0738	0.0833	97.78	97.50
6	0.0706	0.0830	98.08	97.60
7	0.0641	0.0817	98.25	97.70
8	0.0622	0.0818	98.50	97.60

Сравнение с лабораторной работой 1

Показатель	CNN	DenseNet121
Точность на тесте	52.49%	96.40%
Ошибка на тесте	1.2763	0.1098
Обучаемые параметры	356810	10250
Число эпох	25	8

DenseNet121 улучшила точность на 43.91 процентного пункта. Разделение STL-10 и оптимизатор одинаковые, но модели отличаются предобучением, размером входа, аугментацией и числом эпох. Поэтому прирост нельзя приписать только архитектуре.

Графики и предсказания на тестовой выборке

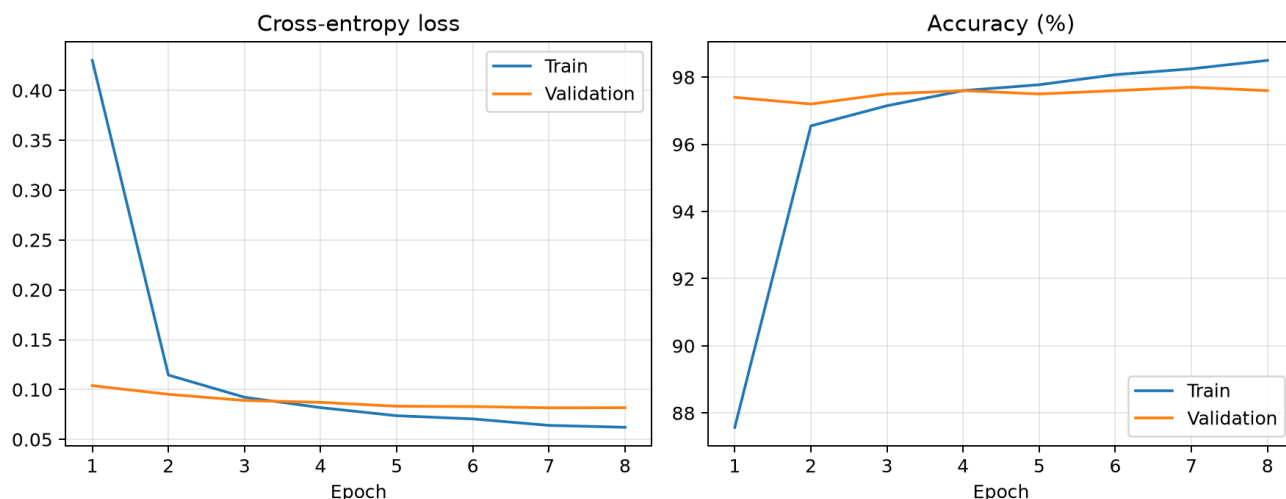


Рисунок 1 Изменение CrossEntropyLoss и точности по эпохам

Train - обучение; Validation - валидация. Случайные аугментации применяются только при обучении. Итоговые тестовые показатели измеряются для сохранённых лучших весов.

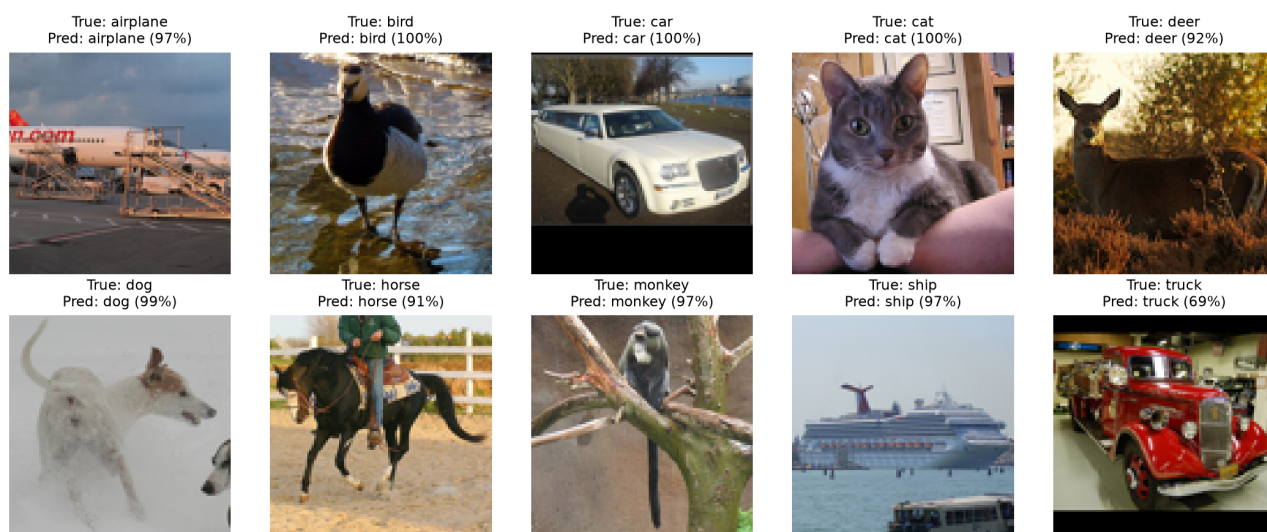


Рисунок 2 По одному тестовому изображению каждого класса

True - настоящий класс; Pred - ответ модели. Показаны первые изображения каждого класса в исходном порядке тестовой выборки, без отбора только правильных ответов. Процент в скобках - значение softmax для предсказанного класса.

Распознавание произвольного изображения

Одна внешняя фотография dog.jpg подаётся на обе сети. Для каждой применяется собственная предобработка: 96×96 и нормализация 0.5 для CNN, масштабирование до 256, центральная обрезка 224×224 и нормализация ImageNet для DenseNet121.

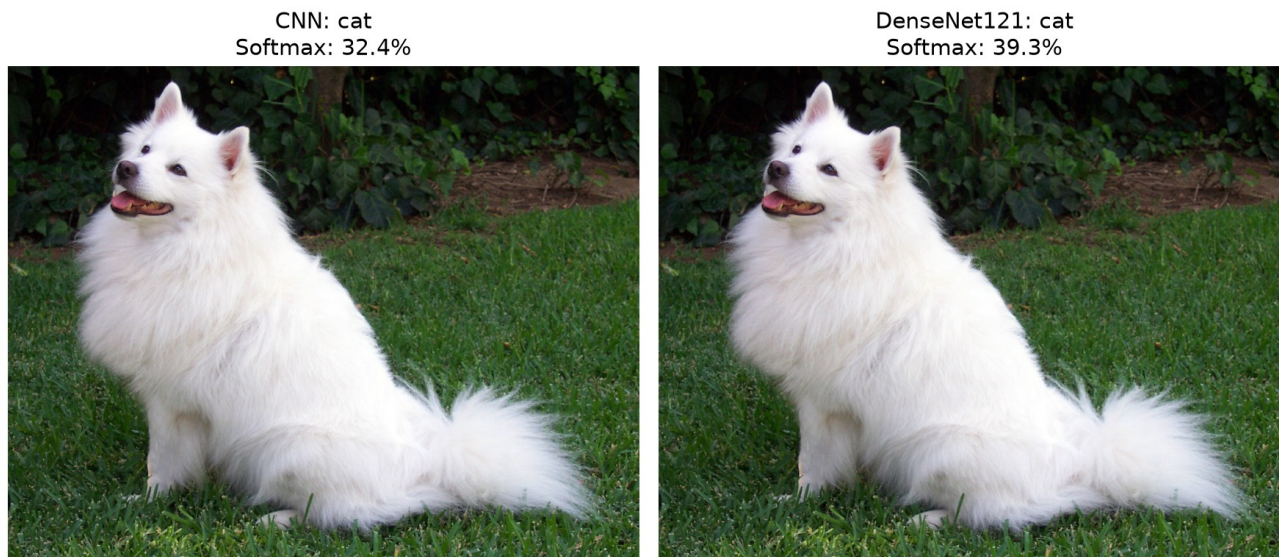


Рисунок 3 Сравнение CNN и DenseNet121 на одном изображении

Модель	Ответ	Softmax
CNN	cat	32.37%
DenseNet121	cat	39.28%

На фотографии изображена собака; обе сети ошибочно выбрали класс cat. Этот пример показывает, что высокая тестовая точность не гарантирует правильность ответа на внешний файл. Общая эффективность определяется точностью на всех 8000 тестовых изображениях. Softmax показывает уверенность модели и не гарантирует правильность ответа.

Запуск программы

```
python reports/Dulko/lab2/src/lab2.py --image reports/Dulko/examples/dog.jpg
```

Параметр --choose открывает окно выбора файла. Результат сохраняется в папке results в формате PNG; обученные веса загружаются без повторного обучения. Полные команды установки и запуска приведены в README.md.

Сопоставление с опубликованными SOTA результатами

Метод	Точность	Условия
MixMatch [4]	94.41%	5000 меток и неразмеченные данные
ViT-L/16 + Spinal FC + Background [5]	$99.71 \pm 0.06\%$	Предобучение и дополнительные фоновые изображения

Для MixMatch ошибка 5.59% переведена в точность: $100 - 5.59 = 94.41\%$. В статье Kabir результат ViT обозначен как SOTA на май 2023 года. Эти числа являются опубликованными ориентирами, а не утверждением о текущем абсолютном рекорде.

Условия исследований отличаются от лабораторной: используются более мощные сети, внешнее предобучение или дополнительные данные. В этой работе CNN обучается с нуля, а DenseNet использует ImageNet; обучение на STL-10 ограничено 4000 размеченных изображений. Прямое сравнение с SOTA не является сравнением при одинаковых условиях.

Вывод

Предобученная DenseNet121 адаптирована к 10 классам STL-10 заменой классификатора и обучена с SGD. Тестовая точность 96.40% выше результата CNN 52.49% на 43.91 процентного пункта. Предобученные признаки полезны при ограниченном числе размеченных изображений. Реализована визуализация ответов обеих сетей на одном файле и проведено сопоставление с опубликованными результатами.

Источники

1. STL-10, описание данных и протокол оценки
<https://cs.stanford.edu/~acoates/stl10/>
2. Torchvision, STL10
<https://docs.pytorch.org/vision/0.23/generated/torchvision.datasets.STL10.html>
3. Torchvision, DenseNet121 и предобработка
<https://docs.pytorch.org/vision/0.23/models/generated/torchvision.models.densenet121.html>
4. Berthelot et al. MixMatch, NeurIPS 2019, таблица 2
<https://papers.nips.cc/paper/2019/file/1cd138d0499a68f4bb72bee04bbec2d7-Paper.pdf>
5. Kabir. Reduction of Class Activation Uncertainty with Background Information, таблицы I и III
<https://arxiv.org/abs/2305.03238>
6. Фотография собаки, PyTorch Hub
<https://github.com/pytorch/hub/blob/master/images/dog.jpg>

Дата обращения к источникам 30.09.2026.